

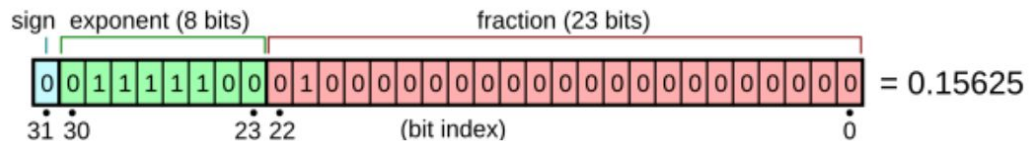
ИНТЕЛЛЕКТ ➤

Инференс, квантизация

к.ф.-м.н. Тихомиров Михаил Михайлович

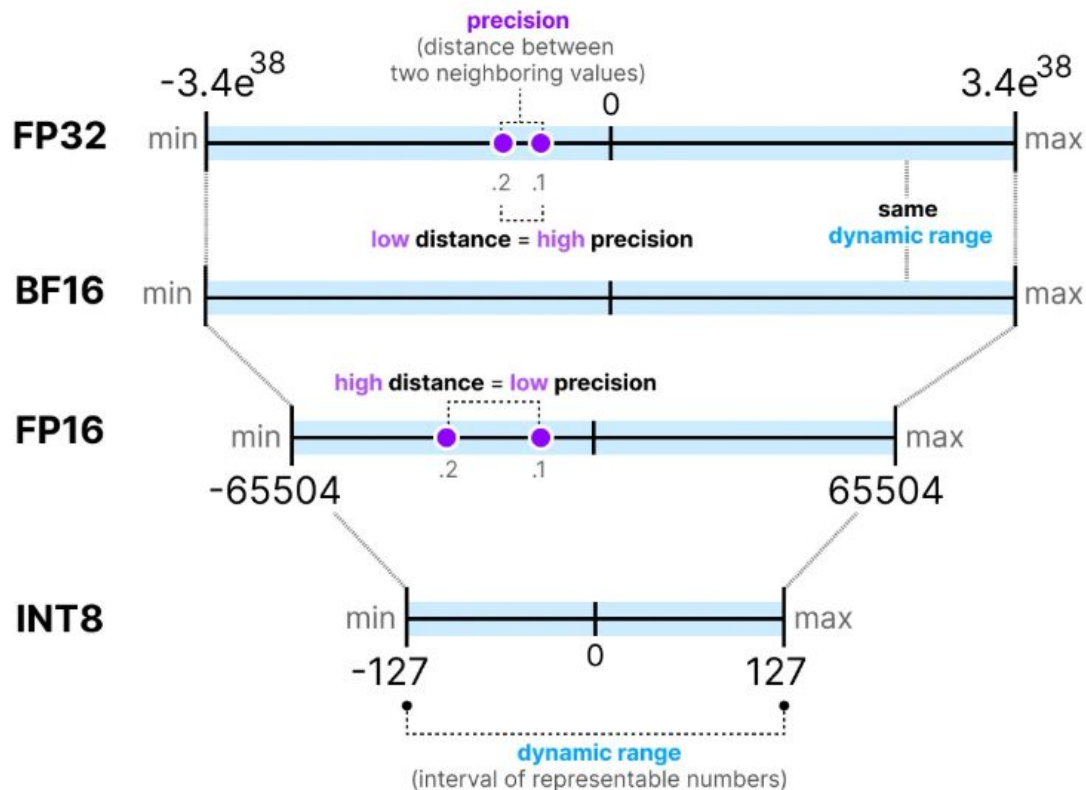
старший научный сотрудник
НИВЦ МГУ имени М. В. Ломоносова

Московский
государственный
университет
имени М.В. Ломоносова



$$\text{value} = (-1)^{\text{sign}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i} \right).$$

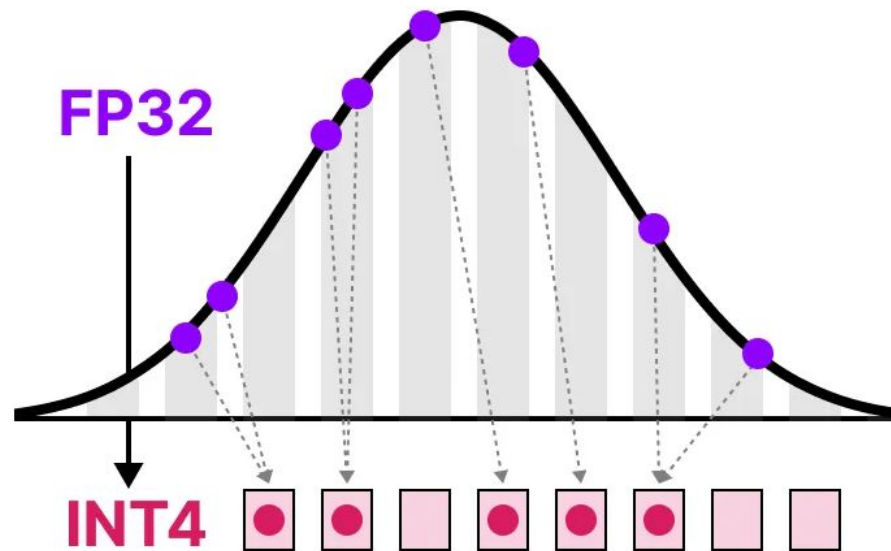
Квантизация: диапазон значений



Квантизация



	Scalar	Vector	Matrix	Tensor
Quantized	1	$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	$\begin{bmatrix} [1 & 2] & [3 & 2] \\ [1 & 7] & [5 & 4] \end{bmatrix}$
High Precision	1.3	$\begin{bmatrix} 1.4 \\ 2.7 \end{bmatrix}$	$\begin{bmatrix} 1.8 & 2.2 \\ 3.7 & 4.2 \end{bmatrix}$	$\begin{bmatrix} [1.1 & 2.7] & [3.75 & 2.7] \\ [1.7 & 7.9] & [5.31 & 4.9] \end{bmatrix}$



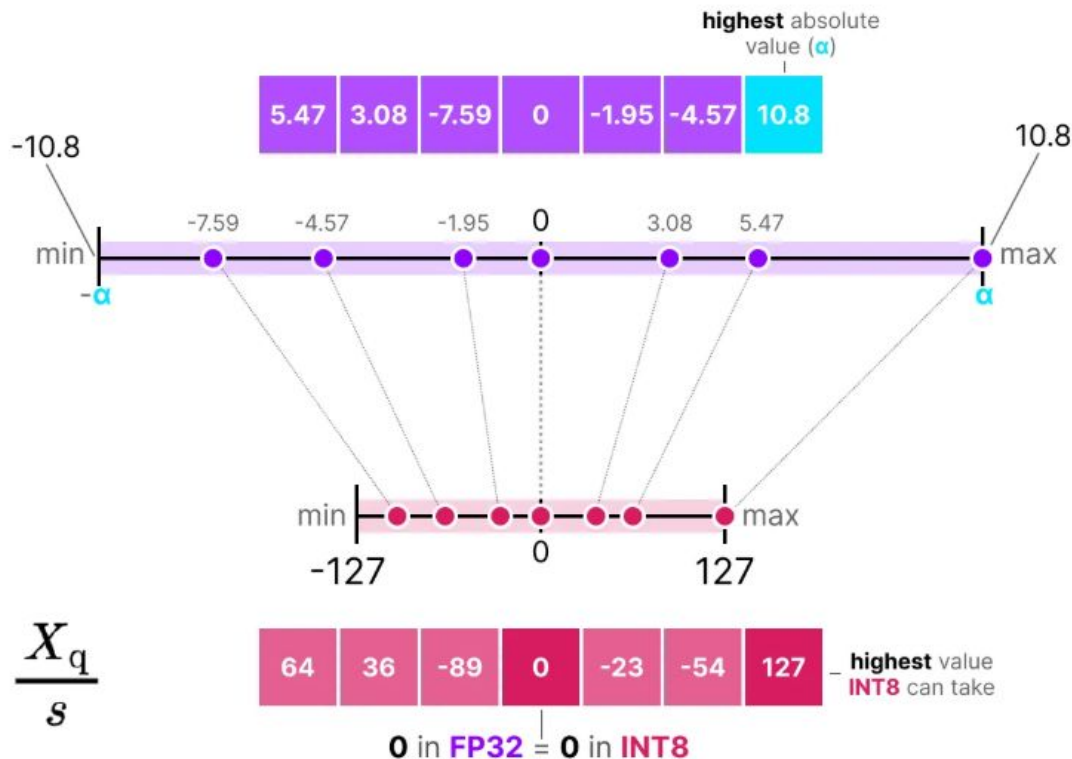
Формула:

$$s = \frac{127}{\max_{i,j} |X_{i,j}|}$$

Квантизация:

$$X_q = \text{round}(s \cdot X)$$

Деквантизация: $X_{dq} = \frac{X_q}{s}$



Zero-point



Формула:

$$\begin{cases} s \min_{i,j} X_{i,j} + z = -128, \\ s \max_{i,j} X_{i,j} + z = 127 \end{cases} \iff \begin{cases} s = \frac{255}{\max_{i,j} X_{i,j} - \min_{i,j} X_{i,j}}, \\ z = -s \min_{i,j} X_{i,j} - 128 \end{cases}$$

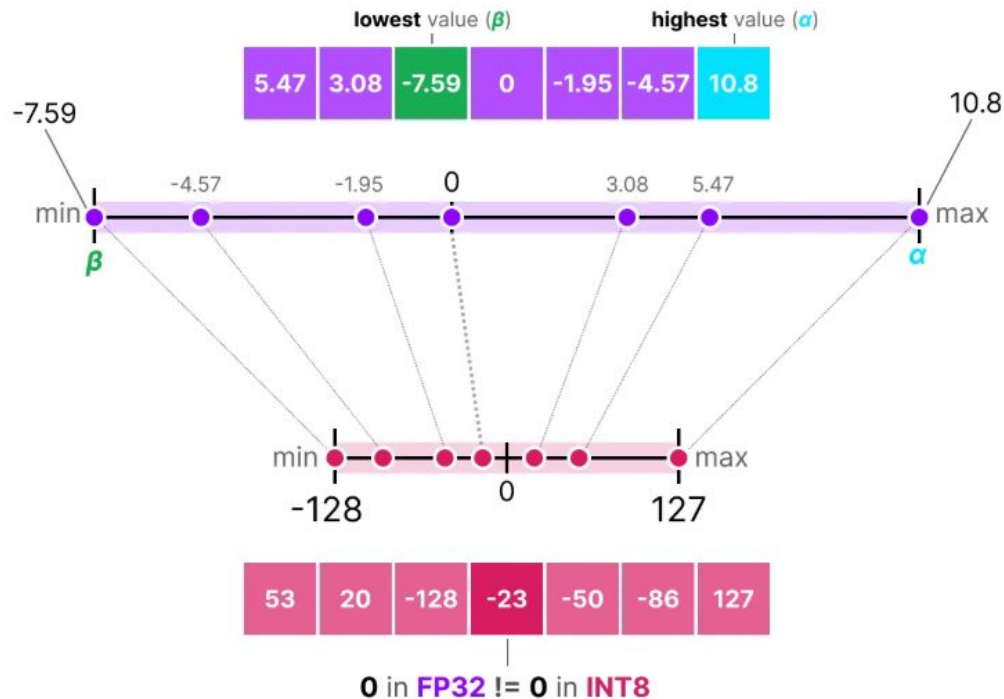
$\tilde{z} = \text{round}(z)$

Квантизация:

$$X_q = \text{round}(s \cdot X + \tilde{z}),$$

Деквантизация:

$$X_{dq} = \frac{X_q - \tilde{z}}{s}$$



Absmax vs zero-point: пример



$$X = \begin{pmatrix} 1.5 & 0 & -1.1 \\ 2 & 0.5 & -0.1 \\ 1.2 & 1.7 & -0.5 \end{pmatrix}$$



absmax

$$X_q = \begin{pmatrix} 95 & 0 & -70 \\ 127 & -2 & 36 \\ 76 & 108 & -32 \end{pmatrix}$$



$$X_{dq} = \begin{pmatrix} 1.496 & 0 & -1.102 \\ 2 & 0.504 & -0.094 \\ 1.197 & 1.701 & -0.504 \end{pmatrix}$$

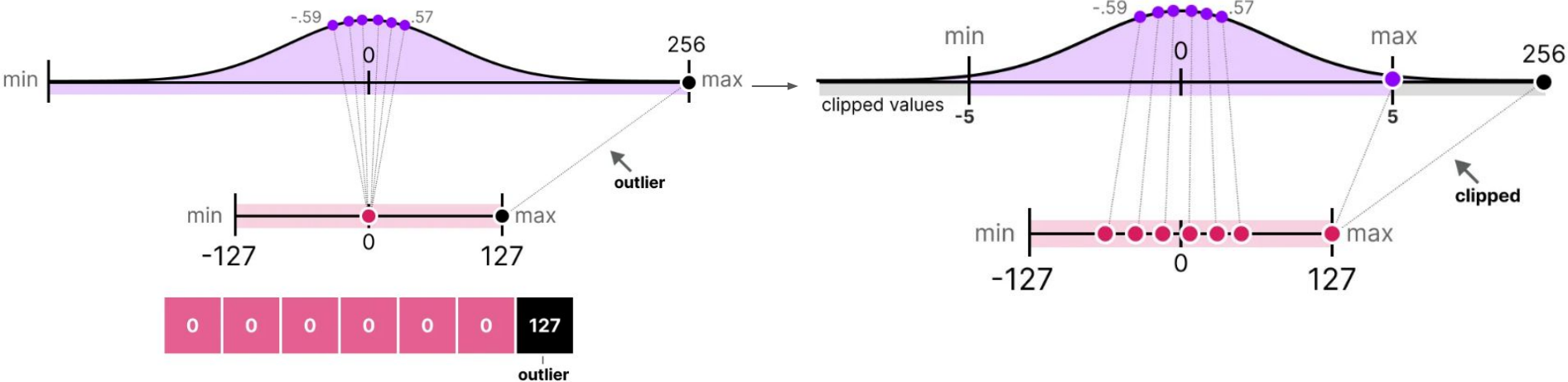
zero-point

$$X_q = \begin{pmatrix} 85 & -38 & -128 \\ 127 & -3 & -46 \\ 61 & 102 & -79 \end{pmatrix}$$

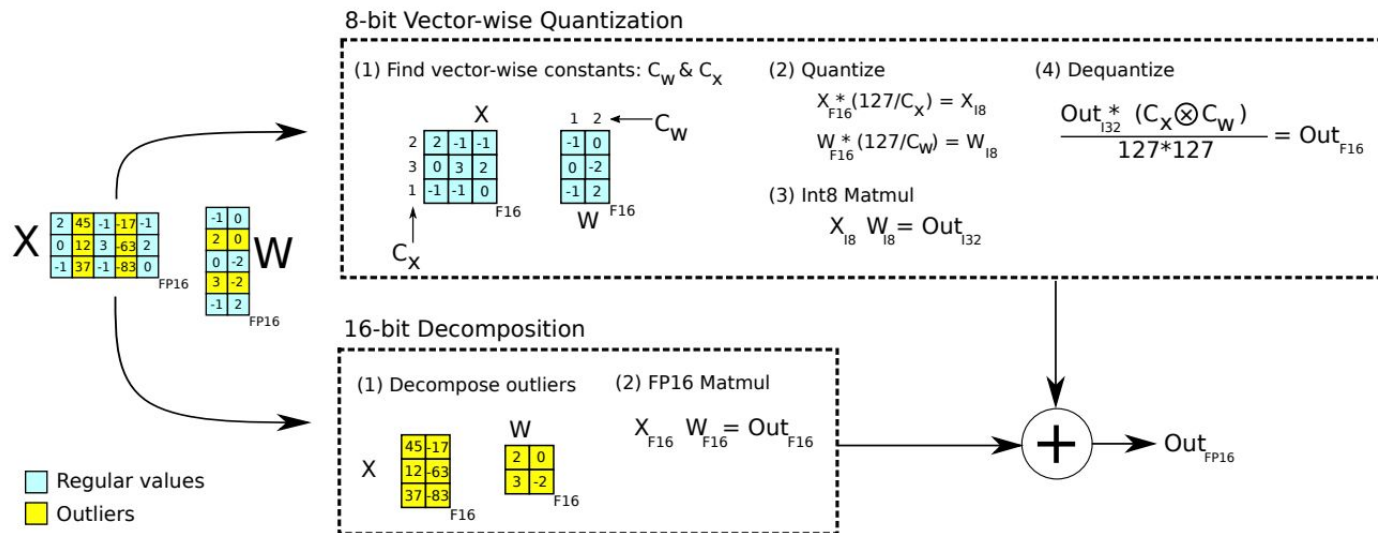


$$X_{dq} = \begin{pmatrix} 1.495 & 0 & -1.094 \\ 2.006 & 0.498 & -0.097 \\ 1.204 & 1.702 & -0.498 \end{pmatrix}$$

Выбросы

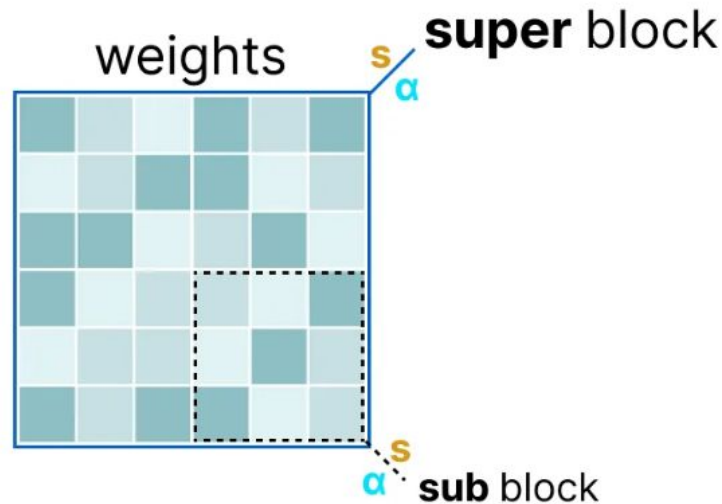


- Основа bnb
- Квантует веса и активации
- vector-wise квантизация
- Отдельная обработка выбросов активаций



- Веса нейросети распределены около 0 и нормально $\rightarrow$ можно нормализовать к среднему = 0 и диапазону $[-1;1]$
- Рассмотрим линейную 4-бит квантизацию, выделив на диапазоне $[-1;1]$ 16 значений:
 - $[-1, -0.8667, -0.7333, \dots, -0.0667, 0.0667, \dots, 0.7333, 0.8667, 1]$
- Находим для нашего числа ближайшее значение в векторе, его индекс и будет квантизированным значением
- NF4 идейно похож, но умнее, числа-константы выбираются на базе статистики весов, например (0.0 к слову нужен и важен)
 - $[-1.0, -0.696, -0.525, -0.395, \dots, -0.091, 0.0, 0.080, \dots, 0.563, 0.723, 1.0]$

- Слои разбиваются на суперблоки
- Каждый блок разбивается на вложенные блоки (sub block)
- Каждый вложенный блок квантуется absmax методом
- NO scale factor каждого вложенного блока также квантуется на уровне суперблока



$$X_{\text{quantized}} = S \cdot X$$

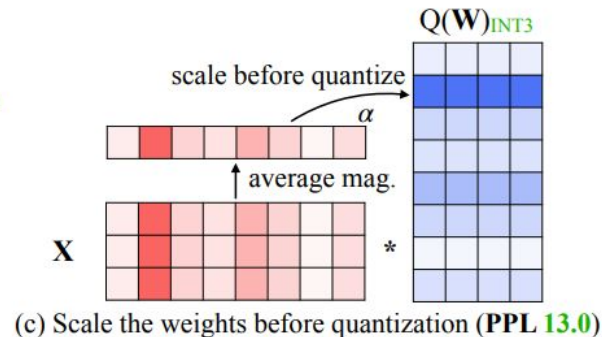
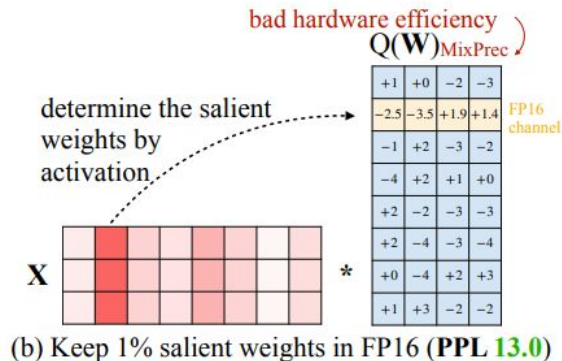
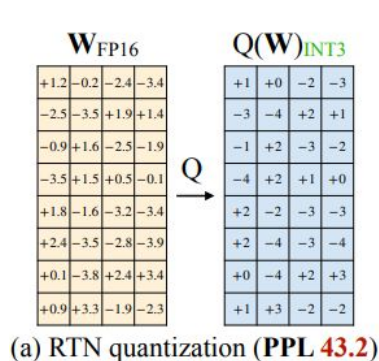
$$X_{\text{quantized}} = \underbrace{S_{\text{sub}}}_{\substack{\text{quantized using} \\ S_{\text{super}}}} \cdot X$$

Используется в llama.cpp и его
клонах!

AWQ: Activation-aware Weight Quantization



- Собирает статистику активаций на некотором небольшом корпусе
- “Все животные равны, но некоторые равнее (с)” - не все веса одинаково важны, причем разница очень существенная.



Используется в vLLM.

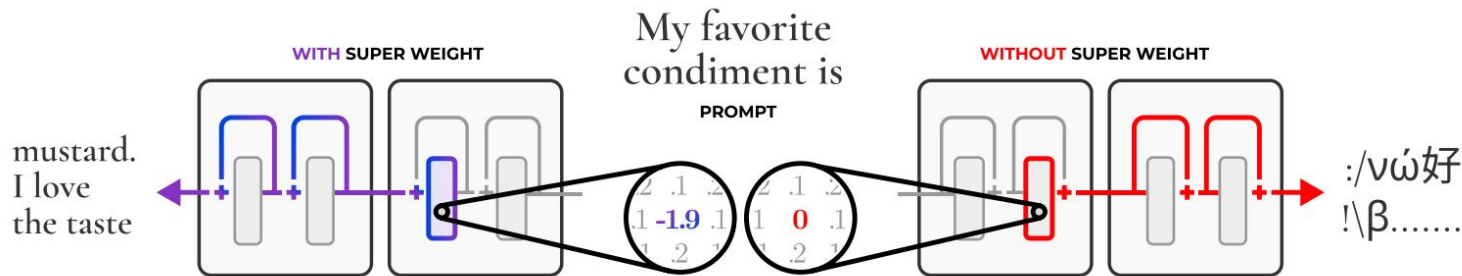
Феномен “супер весов”



Супер веса - веса, которые существенно влияют на поведение модели (одно число в конкретной матрице!)

В исследованных моделях подобных весов около десятка!

Обычно в первых слоях в MLP.



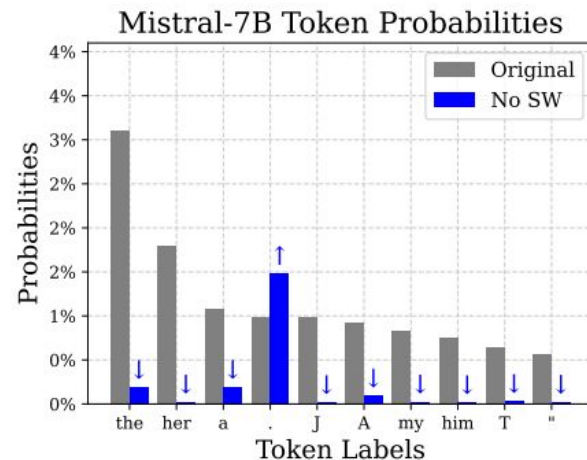
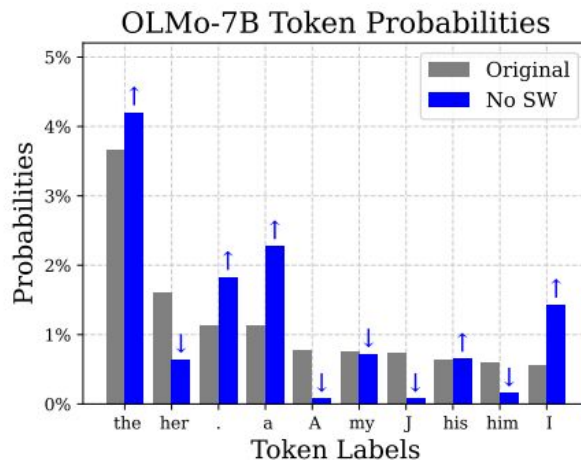
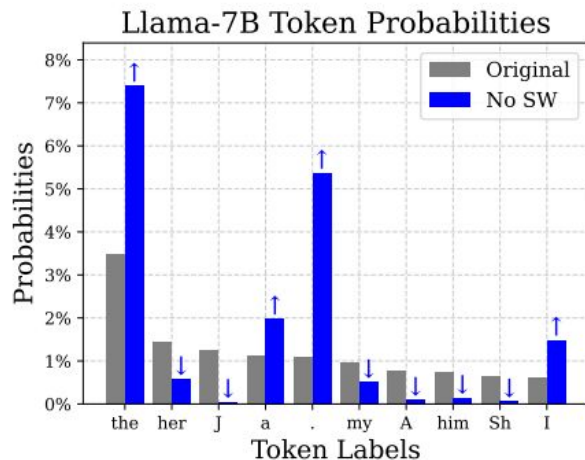
Феномен “супер весов”: деградация качества



Московский
государственный
университет
имени М.В. Ломоносова

Llama-7B	Arc-c	Arc-e	Hella.	Lamb.	PIQA	SciQ	Wino.	AVG	C4	Wiki-2
Original	41.81	75.29	56.93	73.51	78.67	94.60	70.01	70.11	7.08	5.67
Prune SW	19.80	39.60	30.68	0.52	59.90	39.40	56.12	35.14	763.65	1211.11
Prune Non-SW	41.47	74.83	56.35	69.88	78.51	94.40	69.14	69.22	7.57	6.08
Prune SW, +SA	26.60	54.63	56.93	12.79	67.95	61.70	70.01	50.09	476.23	720.57

Феномен “супер весов”: влияние на стоп слова



Инференс

Can i run it LLM?



Can you run it? LLM version

Information

- GPU information comes from [TechPowerUp GPU Specs](#)
- Mainly based on [Model Memory Calculator](#) by hf-accelerate using `transformers` library
- Inference is calculated following [EleutherAI Transformer Math 101](#), where is estimated as

$$\text{Memory}_{\text{Inference}} \approx \text{Model Size} \times 1.2$$

- For LoRa Fine-tuning, I'm assuming a **16-bit** dtype of trainable parameters. The formula (in terms of GB) is

$$\text{Memory}_{\text{LoRa}} \approx \left(\text{Model Size} + \# \text{ trainable Params}_{\text{Billions}} \times \frac{16}{8} \times 4 \right) \times 1.2$$

[NousResearch/Meta-Llama-3-8B-Instruct](#) (7.6B)

int4 int8 float16/bfloat16 float32

int4 refers to models in GPTQ-4bit, AWQ-4bit Or Q4_0 GGUF/GGML

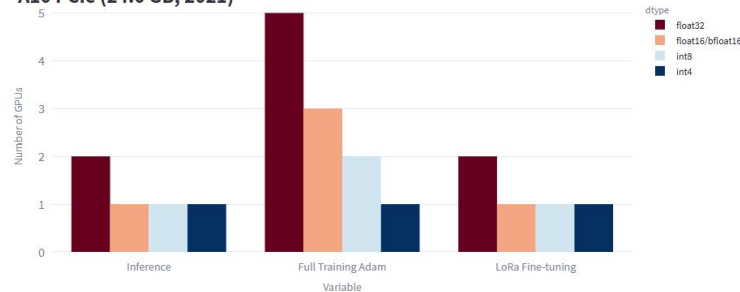
✓ You require 1 GPUs for Inference

✓ You require 1 GPUs for Full Training Adam

✓ You require 1 GPUs for LoRa Fine-tuning (2.0%)

	float32	float16/bfloat16	int8	int4
Total Size (GB)	27.96	13.98	6.99	3.49
Inference (GB)	33.55	16.77	8.39	4.19
Training using Adam (GB)	111.83	55.92	27.96	13.98
LoRA Fine-Tuning (GB)	34.99	18.22	9.83	5.63

Number of GPUs required for A10 PCIe (24.0 GB, 2021)



Через что обычно “генерируют” с LLM



- Huggingface + transformers + pytorch
 - “Нативное” решение, низкая эффективность
- VLLM
 - Один из лучших фреймворков для инференса с нацеленностью на высокую пропускную способность
- SGLang
 - Аналог VLLM, где-то лучше, где-то хуже
- llama.cpp (или ее клоны)
 - olama
 - aphrodite

Какие характеристики оценивают



- Занимаемая память
- Токен/секунда
- Пропускная способность при “нагрузке”
 - Обработка батчами (Batch inference)
 - Токен/секунда или RPS (запрос в секунду)
- Время до первого токена
- Поддерживаемые квантизации
- Возможность multigpu/multinode
- Возможность cpu offload

HF + transformers: инициализация



```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, GenerationConfig

MODEL_NAME = "RefalMachine/RuadaptQwen2.5-1.5B-instruct"
model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    load_in_8bit=False,
    torch_dtype=torch.bfloat16,
    device_map="cuda:0",
    attn_implementation='flash_attention_2'
)
model.eval()

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
generation_config = GenerationConfig.from_pretrained(MODEL_NAME)
generation_config.max_new_tokens = 512
generation_config.temperature = 0.3
generation_config.repetition_penalty = 1.05
```

HF + transformers: генерация



```
import time
inputs = ["Напиши функцию для генерации ответа через LLM, используя transformers."]
for query in inputs:
    prompt = tokenizer.apply_chat_template([
        {
            "role": "user",
            "content": query
        }
    ], tokenize=False, add_generation_prompt=True)

    data = tokenizer(prompt, return_tensors="pt", add_special_tokens=False)
    data = {k: v.to(model.device) for k, v in data.items()}

    s = time.time()
    output_ids = model.generate(**data, generation_config=generation_config)[0]
    output_ids = output_ids[len(data["input_ids"][0]):]
    output = tokenizer.decode(output_ids, skip_special_tokens=True).strip()
    gen_time = time.time() - s
    print(len(output) / gen_time)
    print(output)
```

HF + transformers: пример результата



167.16134602604677

Для генерации ответа через LLM (Language Model) с использованием библиотеки `transformers` из PyTorch, вам нужно выполнить следующие шаги:

1. ****Установите необходимые библиотеки****:

```
``bash
pip install transformers torch
``
```

2. ****Создайте функцию генерации ответа****:

```
``python
from transformers import AutoModelForCausalLM, AutoTokenizer

def generate_response(prompt):
    # Устанавливаем модель и токенайзер
    model_name = "EleutherAI/gpt-neo-125M"
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(model_name)

    # Предлагаемый запрос
    input_text = prompt

    # Преобразуем текст в токены
    inputs = tokenizer.encode(input_text, return_tensors='pt')

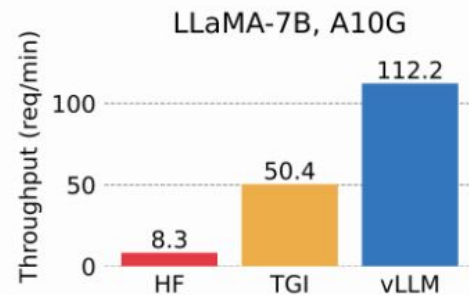
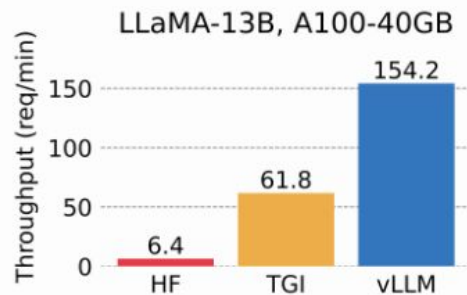
    # Генерируем ответ
    with torch.no_grad():
        outputs = model.generate(inputs, max_length=50, num_return_sequences=1)

    # Переводим ответ обратно в текст
    response_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

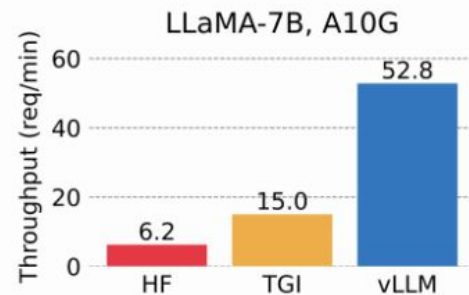
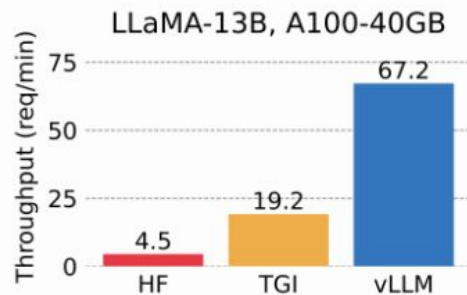
    return response_text

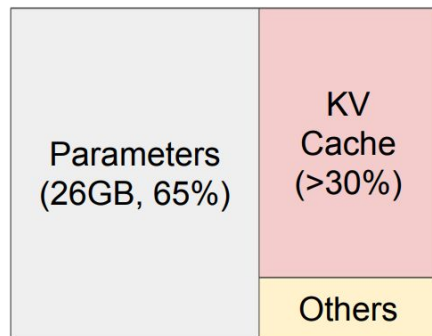
# Пример использования
prompt = "Какой будет следующий шаг в разработке проекта?"
print(generate_response(prompt))
``
```

- Одно из главных узких мест инференса LLM - память
- В частности классический attention + kv cache потребляет много (1.7GB for a single sequence in LLaMA-13B)
- Реальное ускорение не такое большое, как на графике

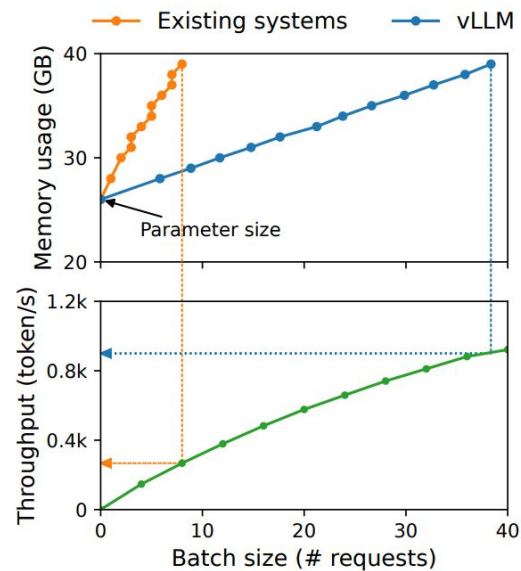


Serving throughput when each request asks for *one output completion*. vLLM achieves 14x - 24x higher throughput than HF and 2.2x - 2.5x higher throughput than TGI.





NVIDIA A100 40GB



KV cache

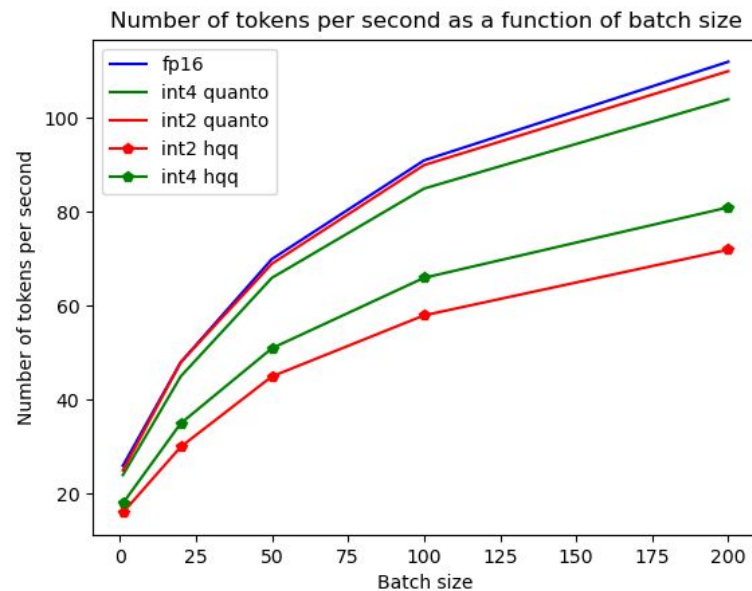
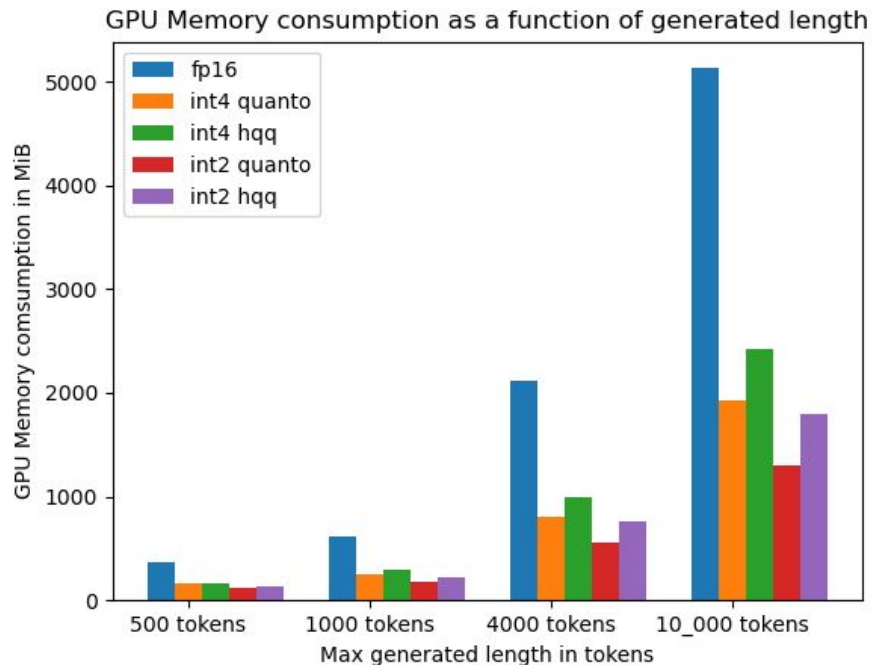


- Мы генерируем токен за токеном, высчитывая на каждом слое $Q K V \dots$
- K и V не меняются для будущих токенов, так как зависят только от прошлых, и они нужны для расчета будущих токенов
- Их можно закешировать!

Для одного токена требуется байт (в fp16)

$$2 \cdot 2 \cdot n_{\text{layers}} \cdot n_{\text{heads}} \cdot d_{\text{head}}$$

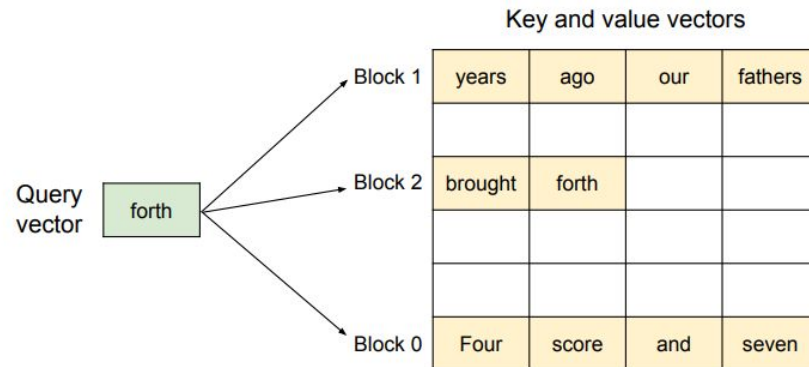
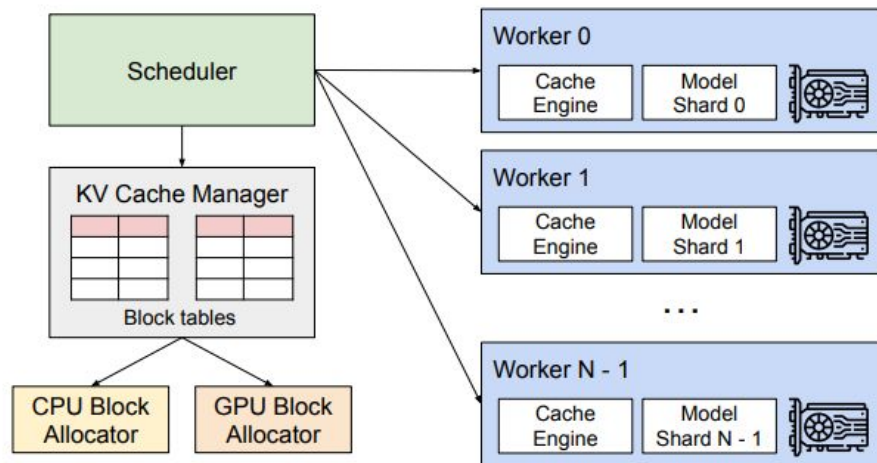
KV cache: квантизированный





- Одна из основных фич vLLM
- “Наивный HF” работает с kv-cache не эффективно
- Разбивает kv-cache на части, эффективно их хранит и обращается по аналогии с **страничной организацией памяти** в ОС
- При одинаковых префиксах (промптах) и множественном декодировании, для разных последовательностей общая часть хранится только в едином экземпляре
- Во время расчета внимания, PagedAttention определяет и извлекает разные блоки KV-кеша по отдельности.

Paged Attention



VLLM: инициализация



```
from vllm import LLM, SamplingParams
from transformers import AutoTokenizer

MODEL_NAME = "RefalMachine/RuadaptQwen2.5-1.5B-instruct"
sampling_params = SamplingParams(
    temperature=0.01,
    top_p=0.9,
    top_k=40,
    max_tokens=1000,
    repetition_penalty=1.0,
)
llm = LLM(
    model=MODEL_NAME,
    max_seq_len_to_capture=4096,
    max_model_len=4096
)
tokenizer = llm.get_tokenizer()
```

VLLM: генерация



```
import time
inputs = ["Напиши функцию для генерации ответа через LLM, используя transformers."]
for query in inputs:
    messages = [{
        "role": "user",
        "content": query
    }]
    prompt = tokenizer.apply_chat_template(
        messages, tokenize=True, add_generation_prompt=True
    )
    s = time.time()
    outputs = llm.generate(prompt_token_ids=[prompt], sampling_params=sampling_params)
    gen_time = time.time() - s
    for output in outputs:
        generated_text = output.outputs[0].text
        print(len(generated_text) / gen_time)
        print(generated_text)
```

VLLM: пример результата



866.654579678111

Для генерации ответа через LLM (Language Model) с использованием библиотеки `transformers` из PyTorch, вам нужно выполнить следующие шаги:

1. **Установите библиотеки**:

```
```bash
pip install transformers torch
```
```

2. **Создайте функцию генерации ответа**:

```
```python
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

def generate_response(prompt, model_name='qwen', max_length=50):
 # Загрузка модели и токенизатора
 model = AutoModelForCausalLM.from_pretrained(model_name)
 tokenizer = AutoTokenizer.from_pretrained(model_name)

 # Предварительная подготовка данных
 prompt = prompt.strip()
 inputs = tokenizer(prompt, return_tensors='pt')

 # Генерация ответа
 with torch.no_grad():
 outputs = model.generate(**inputs, max_length=max_length, num_return_sequences=1)

 # Преобразование ответа в текст
 response = tokenizer.decode(outputs[0], skip_special_tokens=True)

 return response

Пример использования
prompt = "Какой будет следующий сезон сериала 'Звездные войны'?"
response = generate_response(prompt)
```

# VLLM: openai service



```
from openai import OpenAI

oclient = OpenAI(api_key='123', base_url=url)

myprompt = '''**Instruction**
Ответь на вопрос, но строго следуй формату:

<thoughts>
Твои внутренние размышления на русском (пользователь их не видит)
</thoughts>
<answer>
Ответ для пользователя
</answer>

Question
Сколько букв а в слове балалайка?'''
res = oclient.chat.completions.create(
 model=model_name,
 messages=[{"role": "system", "content": "You are a helpful and harmless assistant. You should think step-by-step."},
 {"role": "user", "content": myprompt}],
 temperature=0.01,
 top_p=0.9,
 max_tokens=1000,
 extra_body={
 "repetition_penalty": 1.0,
 "add_generation_prompt": True,
 }
)
if res.choices is not None:
 res = res.choices[0].message.content.strip()
print(res)
```

```
python -u -m vllm.entrypoints.openai.api_server \
--model /workdir/data/models/qwen/QwQ-32B-Preview \
--gpu-memory-utilization 0.9 \
--max_model_len 8000 \
--max_num_seqs 100 \
--port 8888
```



# VLLM: openai service: пример ответа



<thoughts>

Пользователь спрашивает, сколько букв "й" в слове "балалаайка". Сначала нужно понять, как правильно написано это слово. Обычно слово "балалайка" пишется без extra "й", но возможно здесь опечатка или особенное написание. Проверим правописание слова "балалайка". Да, стандартное написание – балалайка, с одной "й". Если пользователь написал "балалаайка", то возможно, это опечатка, и ли он хочет специально вставить extra "й". Нужно уточнить, но сначала можно посчитать количество "й" в слове "балалаайка", как написано пользователем.

Слово "балалаайка" состоит из следующих букв: б, а, л, а, л, а, а, й, к, а. Давайте перечислим все "й": видим только одну "й" перед "к". Поэтому, в этом слове одна буква "й".

Но, возможно, пользователь имел в виду стандартное слово "балалайка". В этом случае, слово "балалайка" содержит две буквы "й": первая "й" между "л" и "а", вторая "й" перед "к". Но в данном случае пользователь написал "балалаайка", что может быть опечаткой.

Чтобы быть точным, нужно уточнить, но сначала можно ответить на основании того, как написано.

</thoughts>

<answer>

В слове "балалаайка", как вы его написали, буква "й" встречается один раз. Однако, если вы имели в виду стандартное слово "балалайка", то там две буквы "й". Пожалуйста, уточните, если нужно.

</answer>

# Offline Batched inference: сравнение



- Случай с длинной генерацией - огромное ускорение
- В случае генерации 1 токена, ускорения подобного уровня нет

```
import time
prompt_token_ids = [tokenizer(d, add_special_tokens=False)['input_ids'] for d in dataset]
s = time.time()
outputs = llm.generate(prompt_token_ids=prompt_token_ids, sampling_params=sampling_params)
gen_time = time.time() - s
total_len = 0
total_tokens = 0
for output in outputs:
 generated_text = output.outputs[0].text
 total_len += len(generated_text)
 total_tokens += len(output.outputs[0].token_ids)
print(total_len / gen_time)
print(total_tokens / gen_time)
```

Processed prompts: 100%|██████████| 500/500 [00:31<00:00, 16.13it/s, est. speed input: 459.52 toks/s, output: 5921.70 toks/s]

24127.652771122684  
5912.201165616972

```
import time
from tqdm import tqdm
bs = 16

total_len = 0
s = time.time()
for i in tqdm(range(0, len(dataset[:100]), bs)):
 res = model.generate_batch(dataset[i:i+bs])
 batch_char_len = sum([len(t) for t in res[1]])
 total_len += batch_char_len
total_time = time.time() - s

print(total_len / total_time)
```

100%|██████████| 7/7 [02:14<00:00, 19.22s/it]

1259.603208250194

# Multi-gpu/Multi-node



vLLM позволяет очень легко поднять модель в multi-gpu с использованием tensor parallel или pipeline parallel, передав флаги, например:

- tensor-parallel-size 4
- pipeline-parallel-size 2

Предусмотрена возможность multi-node.

vLLM поддерживает квантизации: GPTQ, AWQ,

- Аналог vLLM
- Что-то появляется в нем раньше, чем в vLLM, что-то наоборот
- В среднем “чуть” быстрее 5-10%
- Однако есть серия известных багов с зацикливаниями моделей, которые были отмечены в `habr` публикации от Сбербанка

# Баг в SGLang



“Во время отладки циклов и замеров метрик мы заметили интересную особенность: при повышении DP количество циклов у моделей (причём не только наших!) росло, а скорость инференса значительно падала. После небольшого расследования выяснилось, что мы нашли критичный баг в SGLang, который проявляется при  $dp > 1$  в сочетании с batched-запросами к API. На всех версиях, начиная с **0.5.3** вплоть до актуальной **0.5.9**, модель **начинает зацикливаться**, повторяя куски генераций, а вычисления дублируются на всех DP-рангах, что искажает результаты замеров. Особенно это было заметно в кодовых задачах, где на Qwen3-8B **результаты на HumanEval падали с 0,63 аж до 0,43**. Кроме того, этот баг сводит на нет выигрыш от горизонтального масштабирования на одной ноде, а в случае небольших моделей с DP=8 доля лишнего вычисления в общем времени выполнения становится критической.”

- Другой популярный фреймворк для инференса LLM
- Фокус не на пропускную способность, но на возможность запуска в целом:
  - Свои типы для квантизации GGUF
  - Возможность выгрузки слоев на CPU (cpu offload)
  - Множество форков и успешных “клонов” (ollama и др)

# llamacpp



```
root@e735b069dcc2:/workdir/projects/llama.cpp# CUDA_VISIBLE_DEVICES=0 ./llama-cli -m RuadapTQwen2.5-32B-instruct-GGUF/Q4_K_M.gguf -n 512 -cnv -p "You are Qwen, created by Alibaba Cloud. You are a helpful assi
```

0	NVIDIA A100-SXM4-80GB	Off	00000000:07:00.0 Off	0
N/A	26C P0	74W / 400W	7766MiB / 81920MiB	0% Default Disabled

```
system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.
```

> Привет, как запустить llama-cpp cli с offload половины слоев на CPU?

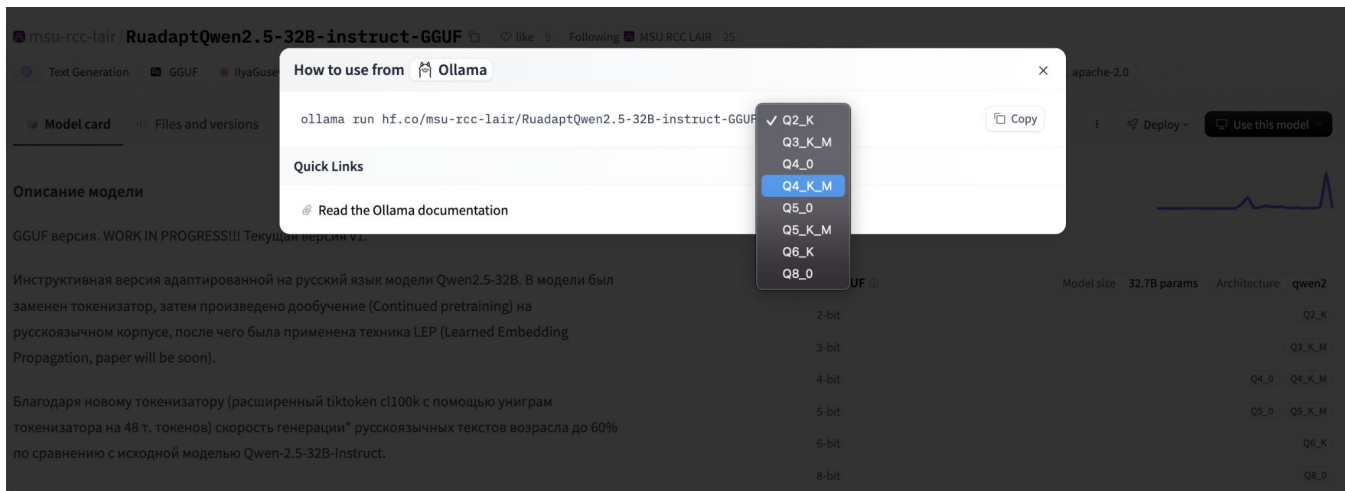
Привет! Для запуска Llama.cpp с offload (переключением нагрузки) половины слоев на CPU, нужно воспользоваться следующими параметрами при запуске CLI версии Llama.cpp. Важно помнить, что точные параметры и поддержка могут зависеть от версии Llama.cpp, которую вы используете. Вот пример команды для запуска с offloading половины слоев на CPU:

```
```bash
./main -m your_model.bin -ngl 1
```
```

В этом примере:

- `-m your_model.bin` - указывает путь к вашему моделируемому файлу.
- `-ngl 1` (Number of GPU Layers) - указывает количество слоев, которые остаются на GPU, при этом остальные слои будут автоматически offloaded на CPU.

- Удобный и популярный клон Llamasrpp
- Можно тянуть GGUF модели прямо с HF





## ollama



```
>>> Чем ollama отличается от llama-cpp?
```

```
Ollama и **LLaMA-CPP** – это две разные реализации, но обе работают с моделями семейства LLaMA.
```

```
Ollama:
```

- Это более новый проект.
- Разработанный как замена для `llama.cpp`, он стремится упростить работу с модели LLM (Large Language Model).
- Ориентирован на удобство и производительность: включает в себя дополнительные функции, такие как автоматическое управление моделью, возможность быстрого переключения между различными моделями LLaMA и поддержка других моделей.
- Написан на Rust и C++, что делает его относительно более быстрым и безопасным с точки зрения памяти.
- Позволяет работать через простой API или командную строку.

```
LLaMA-CPP:
```

- Это первая известная реализация, поддерживающая моделирование модели LLaMA с использованием только CPU (без поддержки GPU).
- Написано на чистом C++ под руководством команды разработчиков от Facebook Research и сообщества open-source.
- Предоставляет низкоуровневый доступ к функциям модели, что делает его мощным для тех, кто хочет кастомизировать или интегрировать модель в свои собственные приложения напрямую.

В общем, если вы ищете более современный подход с дополнительной функциональностью и удобством использования (особенно через API), Ollama может быть предпочтительнее. Однако, для разработчиков, которым нужна полная гибкость на низкоуровневом уровне или те, кому важен полностью открытый исходный код без дополнительных абстракций, `LLaMA-CPP` всё ещё остаётся отличным выбором.

```
Вывод:
```

- **\*\*Ollama\*\***: Простота использования, дополнительные функции и производительность.
- **\*\*LLaMA-CPP\*\***: Низкоуровневая гибкость, полное управление.